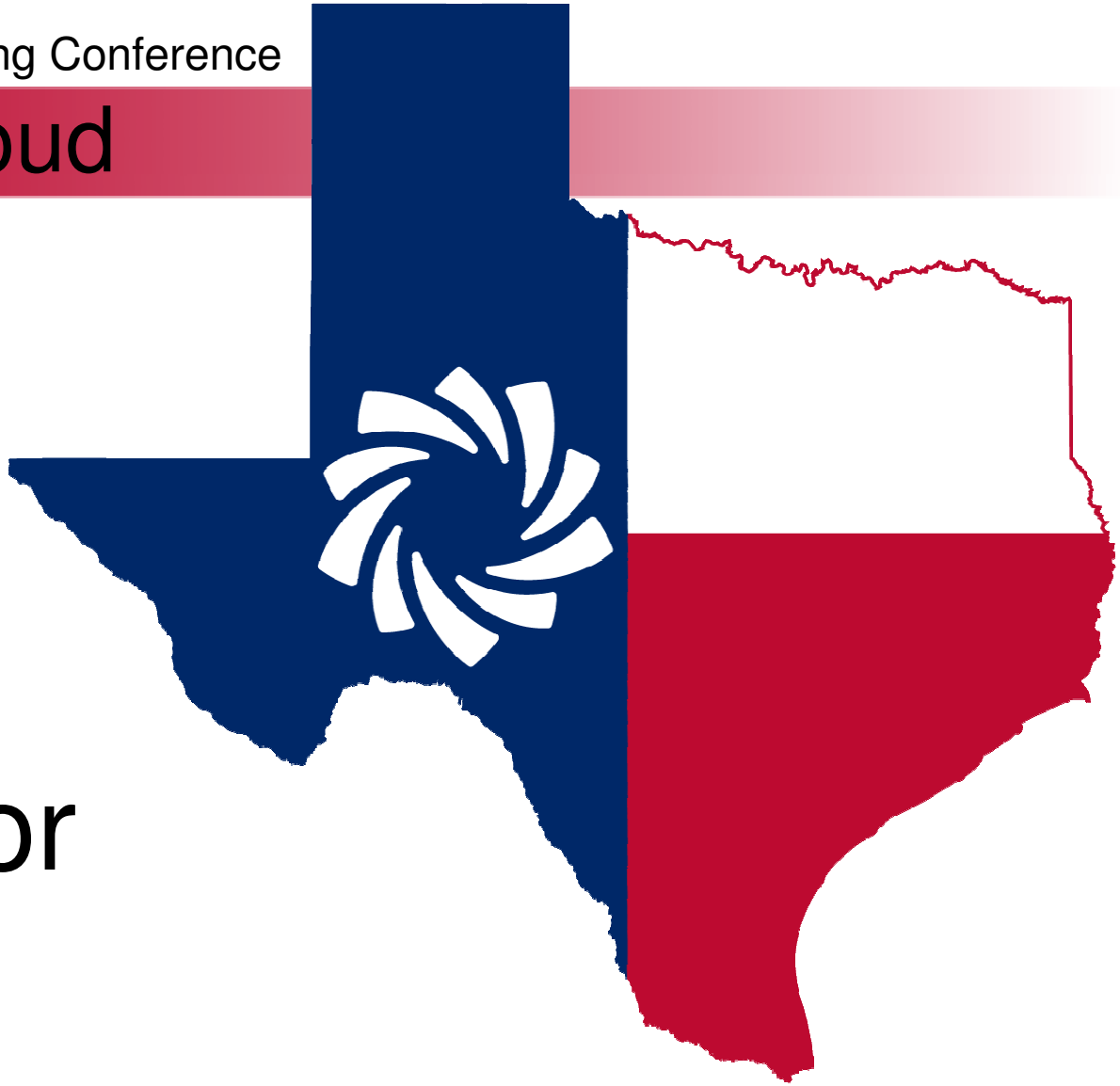
2017 Harris Govern Training Conference

Texas Proud



2017 SQL for Beginners

Friday, November 10, 2017

HARRIS GOVERN

CEU & Session Survey

In order to receive CEU Credits, participants will need to download the Yapp App, select session, then 'Rate' the presentation before the end of the conference.

Simply download the Yapp application from the App Store or Google store, then enter the following code to find the Harris Govern Texas PACS Training Conference: http://my.yapp.us/TXPACS

When downloaded, navigate to:

- 'Multi-track'
- 'VIEW ALL EVENTS' or the specific track (e.g. Track One, Track Two, etc.)
- Select session to attend
- 'Add Event to My Schedule'
- 'RATE' the session attended and provide 'Additional Comments'

This class will cover the anatomy of an SQL statement and how the user can write their own queries using the PACS tables. The client will learn which tables and fields that are commonly used in Appraisal and Collections. The session is structured for users who have no and/or limited SQL knowledge.

Agenda

- Database Basics
- Retrieving DATA
- Sorting Retrieved DATA
- Filtering Retrieved Data
- SQL Query Window
- Demo

DATABASE BASICS

- Definition of a Database
- Tables
- Columns and Data types
- Rows
- Primary Keys

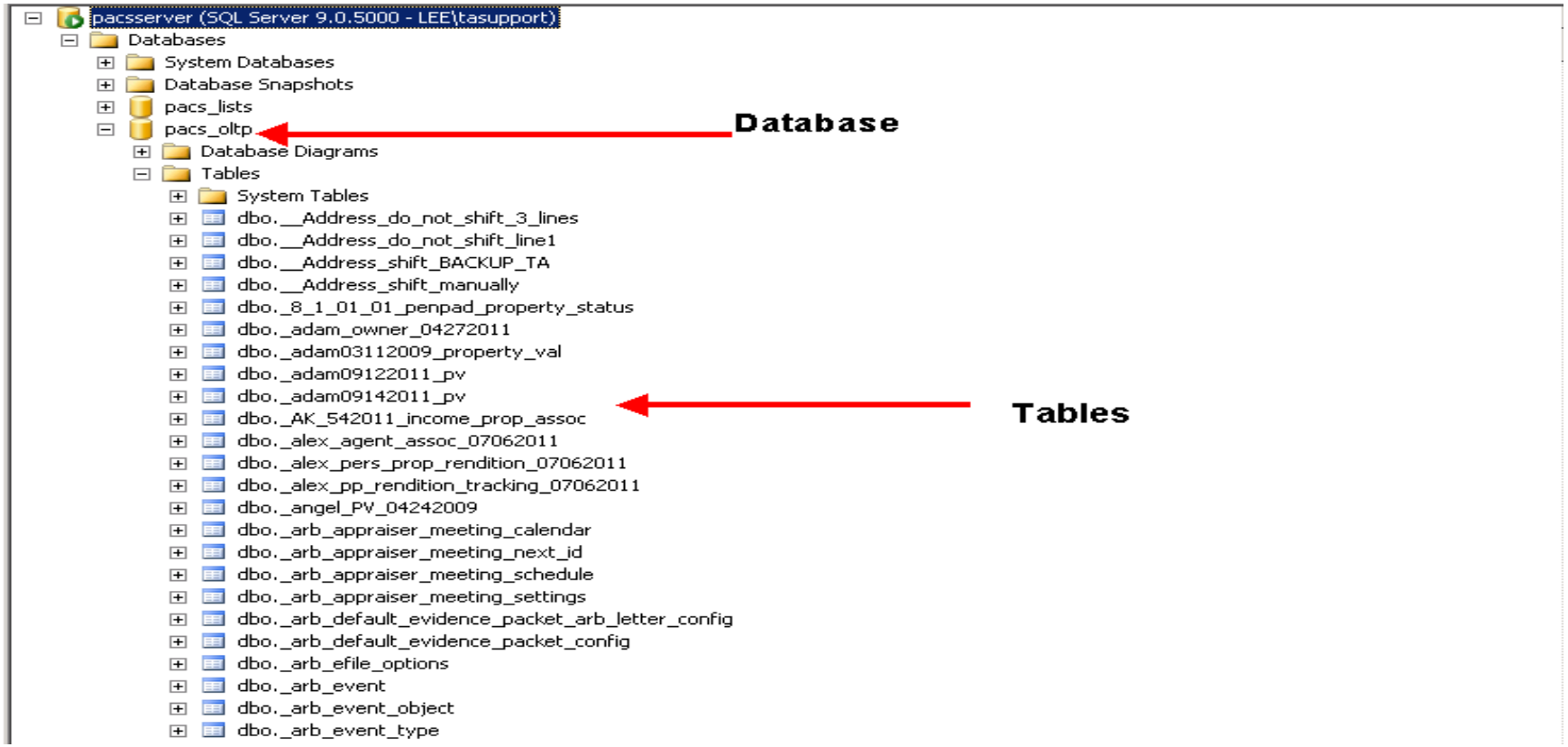
Definition of a Database

A database is a collection of data stored in some organized fashion.

The best way to think of a database is to imagine it being a filing cabinet. The filing cabinet is the database, each drawer inside the filing cabinet is a table and in each drawer contains files which is the data. So that being said; the filing cabinet is simply a physical location to store data.

Tables

A table is a structure file that can store data of a specific type.



Columns and Data Types

•Columns are the fields in which the data is stored. When you are looking to build a query you will enter in the column name of the data that you want to return in your query.

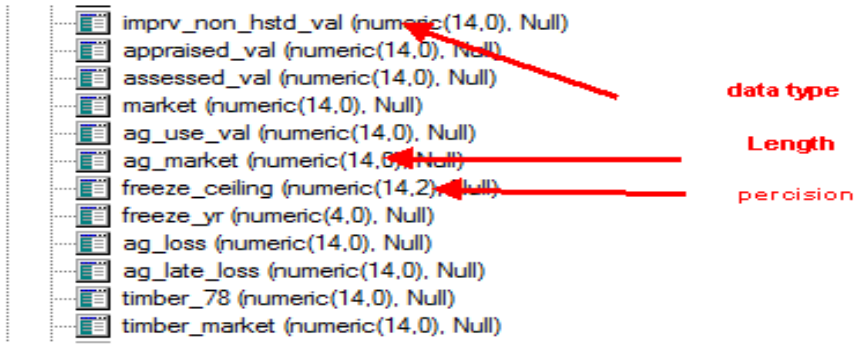
•A column will have a data type. A data type is an attribute that specifies what type of information can be stored in a column, parameter, or variable. Some Base data types are as follows:

- **Char** - Fixed-length non-Unicode character data with a maximum length of 8,000 characters.
- **Varchar** - Variable-length non-Unicode data with a maximum of 8,000 characters.
- **Binary** - Fixed-length binary data with a maximum length of 8,000 bytes.
- **Varbinary** - Variable-length binary data with a maximum length of 8,000 bytes
- **Int** - Integer (whole number) data from -2^31 (-2,147,483,648) through 2^31 - 1 (2,147,483,647).
- **Bit** - Integer data with either a 1 or 0 value.
- **Numeric** - Functionally equivalent to **decimal.**
- **Datetime-** Date and time data from January 1, 1753, through December 31, 9999, with an accuracy of three-hundredths of a second or 3.33 milliseconds.

Columns and Data Types

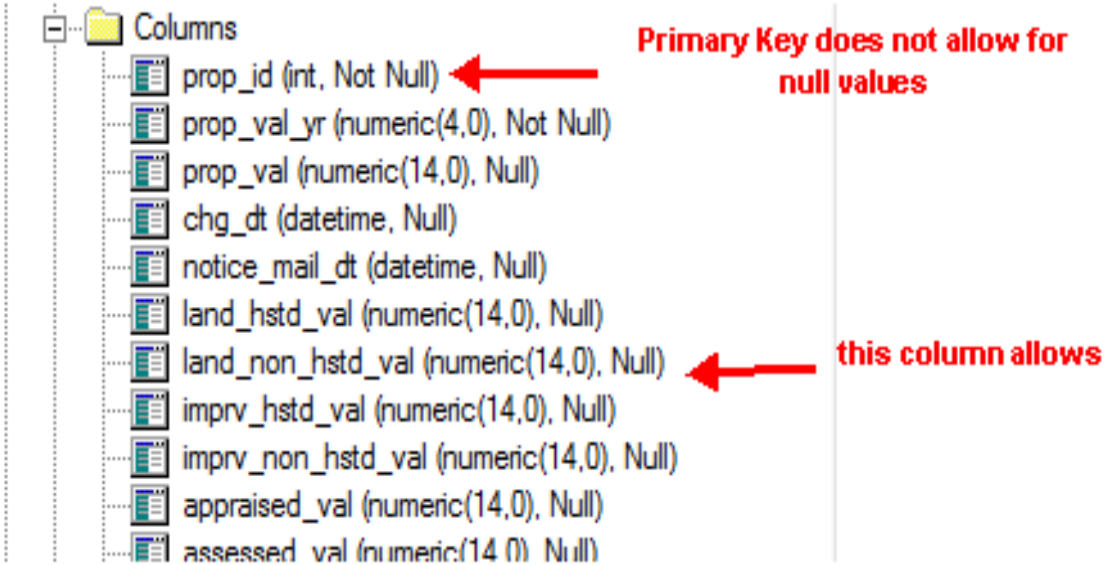
- A column will also contain a length. The length will tell the number of bytes a column can contain. There are certain types of Data types that do not require you to put a length, these are dynamic and they will grow in length as the data grows. Example of these would be int, datetime
- Other data types such as numeric will require you to give it a precision number as well, this is usually done when we are setting up a numeric field, and an example of this would be Land_non_hstd_val it will be a numeric data type, with a length of 14 and precision of 0, or you can see the acreage is setup as numeric with a length of 14 and precision of 4.

Example:



Columns and Data types

•Also when looking at a column you will need to know whether or not a column can contain a null value, remember that all primary keys cannot have a null value. This is important to know, null values will cause a primary key constraint if the field does not all for null values. You can check to see what a column allows by pulling up the table and looking at how the field was designed to be used



Rows

Clipboard | Font | Alignment | Numb

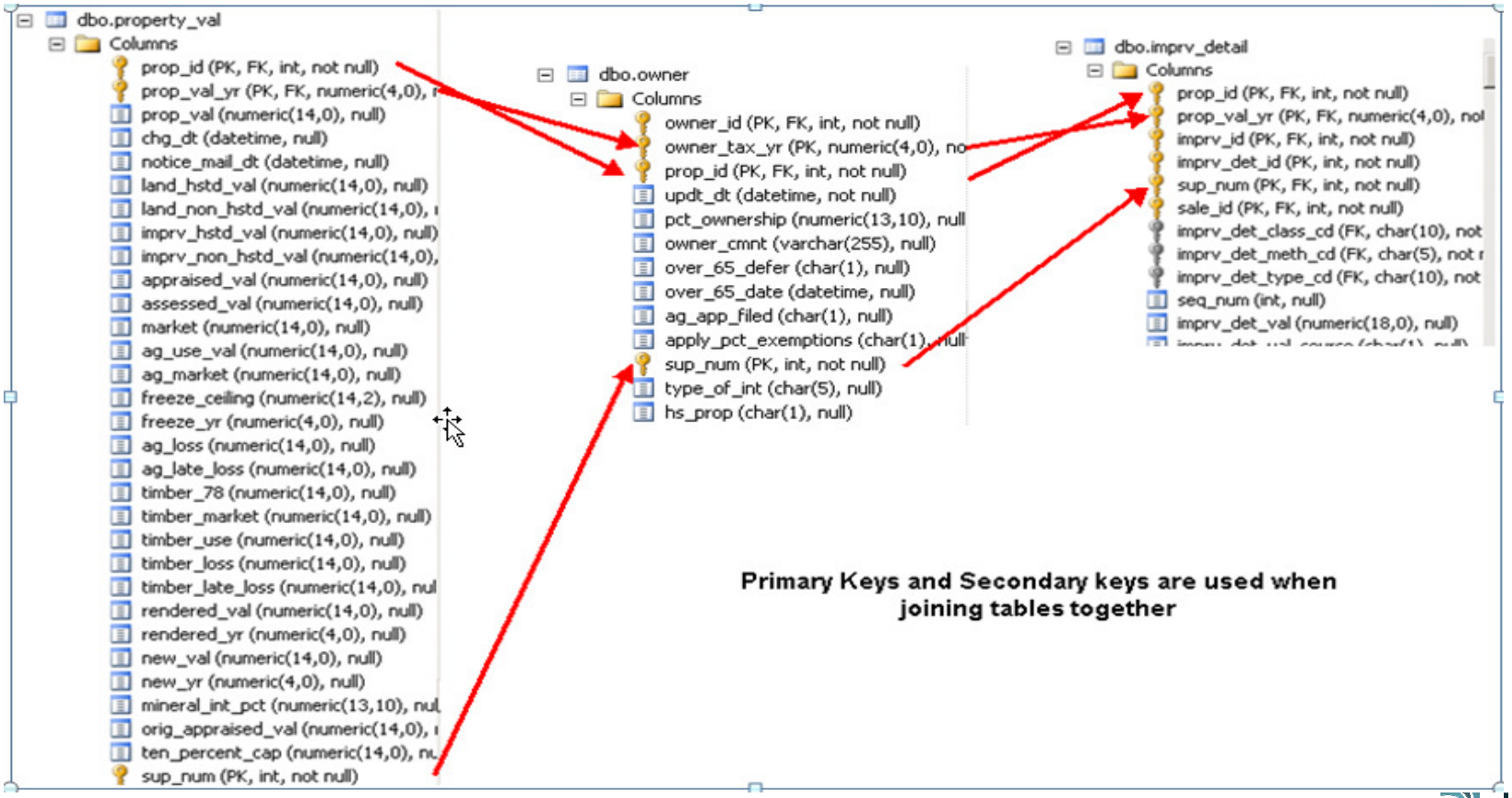
A1 | prop_id

	A	B	C	D	E	F	G
1	prop_id	imprv_id	imprv_adj_amt	imprv_adj_type_cd	land_seg_id	land_value	land_seg_adj_type
2	14160	1687	$ 1.00	DRP			
3	14298	743020	$ (78,878.00)	HIST			
4	14323	2220	$ (15,210.00)	DOR			
5	14337	743042	$ (301,314.00)	HIST			
6	14343				1928	$ (4,958.00)	HIST
7	14343	747791	$ (83,111.00)	HIST			
8	14345				94001	$ (72,000.00)	HIST
9	14345	743050	$ (1,124,422.00)	HIST			
10	14354	735102	$ (641,660.00)	HIST			
11	14412				1996	$ (50,000.00)	HIST
12	14412	2511	$ (189,288.00)	HIST			
13	14412	2511	$ (39,576.00)	HIST			
14	14413	2513	$ (73,059.00)	HIST			
15	14612	3232	$ (75,475.00)	HIST			
16	16085				3946	$ 181,209.00	CONV
17	17771				6011	$ (544,500.00)	DOR
18	17771	743258	$ (3,986,497.00)	DOR			
19	17828	3217	$ (27,999.00)	HIST			
20	17968	743297	$ (157,792.00)	HIST			
21	18073				6370	$ (20,909.00)	DOR
22	18073	72008	$ (90,119.00)	DOR			
23	18278	743396	$ (455,000.00)	MULT			
24	18346				6663	$ (60,480.00)	DOR

- Data in a table is stored in rows: each record saved is stored in its own row. The information stored in each row will contain unique information about that data.
- The columns are the fields in the table.
- The rows contain the data from PACS

Primary Keys

Primary Keys : A column or set of columns that uniquely identify all the rows in a table. Primary keys do not allow null values. No two rows can have the same primary key value; therefore a primary key value always uniquely identifies a single row. More than one key can uniquely identify rows in a table; each of these keys is called a **Secondary Key.**



Retrieving DATA

- Select Statement
- Retrieving Single column
- Retrieving Multiple columns
- Retrieving All Columns

Select Statement

- Lets begin with some basic information and commands that is used to retrieve data. You will need a very simple sentence. You will need to use **SELECT** and **FROM** for you statement.
- **SELECT** - Retrieves rows from the database and allows the selection of one or many rows or columns from one or many tables.
- **FROM** - Specifies the tables, views, derived tables, and joined tables used in SELECT statements.

- The full syntax of the SELECT statement is complex, but the main clauses can be summarized as:

  SELECT [column names] FROM [table_name]

Retrieving Single column

- Lets take a look at a simple statement.

- SELECT **PROP_ID** FROM **PROPERTY_VAL**
- The statement above is returning one column from the table property_val. This statement as no filtering so it will return everything.

- Lets Take a look at what the results will be.

Microsoft SQL Server Management Studio

File Edit View Query Project Tools Window Community Help

New Query | pacs_oltp | Execute

Object Explorer

Connect

- dbo.tnt_export
- dbo.tnt_export_e
- dbo.trans_type
- dbo.transfer_app
- dbo.transfer_app
- dbo.transfer_app
- dbo.transfer_app
- dbo.transfer_app
- dbo.transfer_delc
- dbo.transfer_delc
- dbo.transfer_mt_
- dbo.transfer_mt_
- dbo.transfer_mt_
- dbo.transfer_tax_
- dbo.transfer_tax_
- dbo.transfer_tax_
- dbo.transfer_tax_
- dbo.transfer_tax_
- dbo.transfer_tax_
- dbo.transfer_tax_
- dbo.transfer_tax_
- dbo.UDI_owner_t
- dbo.udi_system_s
- dbo.update_agen
- dbo.update_freez
- dbo.update_owne
- dbo.update_owne
- dbo.user_input_q
- dbo.user_input_q
- dbo.user_list_viev
- dbo.user_rights
- dbo.validate_coll_
- dbo.validation_tol
- dbo.value_cert_n
- dbo.value_cert_n
- dbo.value_cert_n
- dbo.visibility_acce
- dbo.vit_sales
- dbo.websessionst
- dbo.workstation_
- dbo.xsp_pacs_co
- dbo.year_to_date
- dbo.year_to_date
- dbo.year_to_date
- dbo.year_to_date
- dbo.year_to_date
- dbo.ZipAssociatio
- Views

pacsserver.pa...SQLQuery1.sql* | Object Explorer Details

```
select prop_id from property_val
```

Results | Messages

	prop_id
1	1000
2	1001
3	1002
4	1003
5	1004
6	1005
7	1006
8	1007
9	1008
10	1009
11	1010
12	1011
13	1012
14	1013
15	1014
16	1015
17	1016
18	1017
19	1018
20	1019
21	1022
22	1023
23	1024
24	1025
25	1027
26	1028
27	1030
28	1032
29	1033
30	1034
31	1036
32	1037
33	1038
34	1039

Query executed successfully. | pacsserver (9.0 SP4) | LEE\tasupport (63) | pacs_oltp | 00:00:03 | 796358 rows

Ready | Ln 1 | Col 1 | INS

Retrieving Multiple columns

- Lets take a look at a multiple statement.
- SELECT **PROP_ID,PROP_VAL_YR,MARKET** FROM **PROPERTY_VAL**
- The statement above is returning 3 columns from the table property_val. This statement as no filtering so it will return everything.

- Lets Take a look at what the results will be.

Microsoft SQL Server Management Studio

File Edit View Query Project Tools Window Community Help

New Query | pacs_oltp | Execute

Object Explorer

Connect

- dbo.tnt_export
- dbo.tnt_export_e
- dbo.trans_type
- dbo.transfer_app
- dbo.transfer_app
- dbo.transfer_app
- dbo.transfer_app
- dbo.transfer_app
- dbo.transfer_delc
- dbo.transfer_delc
- dbo.transfer_mt_
- dbo.transfer_mt_
- dbo.transfer_mt_
- dbo.transfer_tax_
- dbo.transfer_tax_
- dbo.transfer_tax_
- dbo.transfer_tax_
- dbo.transfer_tax_
- dbo.transfer_tax_
- dbo.transfer_tax_
- dbo.transfer_tax_
- dbo.UDI_owner_t
- dbo.udi_system_s
- dbo.update_agen
- dbo.update_freez
- dbo.update_owne
- dbo.update_owne
- dbo.user_input_q
- dbo.user_input_q
- dbo.user_list_view
- dbo.user_rights
- dbo.validate_coll_
- dbo.validation_tol
- dbo.value_cert_n
- dbo.value_cert_n
- dbo.value_cert_n
- dbo.visibility_acce
- dbo.vit_sales
- dbo.websessionst
- dbo.workstation_
- dbo.xsp_pacs_co
- dbo.year_to_date
- dbo.year_to_date
- dbo.year_to_date
- dbo.year_to_date
- dbo.year_to_date
- dbo.ZipAssociatio
- Views

pacsserver.pa...SQLQuery1.sql* | Object Explorer Details

```
select prop_id,prop_val_yr,market from property_val
```

Results | Messages

	prop_id	prop_val_yr	market
4...	16959	2006	93460
4...	16960	2006	84500
4...	16962	2006	58260
4...	16963	2006	54510
4...	16964	2006	28390
4...	16965	2006	27170
4...	16966	2006	101960
4...	16967	2006	98050
4...	16968	2006	1332220
4...	16969	2006	445900
4...	16970	2006	185425
4...	16971	2006	140485
4...	16972	2006	109010
4...	16973	2006	293940
4...	16974	2006	631380
4...	16975	2006	197480
4...	16976	2006	92150
4...	16977	2006	137185
4...	16978	2006	86890
4...	16979	2006	27570
4...	16981	2006	131985
4...	16982	2006	79360
4...	16983	2006	143190
4...	16984	2006	25420
4...	16986	2006	40850
4...	16987	2006	219550
4...	16988	2006	57730
4...	16989	2006	26930
4...	16991	2006	66875
4...	16992	2006	46940
4...	16993	2006	7230
4...	16994	2006	11250
4...	16995	2006	53150
4...	16997	2006	326350

Query executed successfully. | pacsserver (9.0 SP4) | LEE\tasupport (63) | pacs_oltp | 00:00:04 | 796358 rows

Ready | Ln 1 | Col 1 | INS

Retrieving All columns

- Lets look at a statement that returns all columns
- SELECT * FROM **PROPERTY_VAL**
- The statement above is returns all columns from the table property_val. This statement as no filtering so it will return everything.

- Lets Take a look at what the results will be

Microsoft SQL Server Management Studio

File Edit View Query Project Tools Window Community Help

New Query | Execute

Object Explorer

Connect

- dbo.tnt_export
- dbo.tnt_export_e
- dbo.trans_type
- dbo.transfer_app
- dbo.transfer_app
- dbo.transfer_app
- dbo.transfer_app
- dbo.transfer_app
- dbo.transfer_delc
- dbo.transfer_delc
- dbo.transfer_mt_
- dbo.transfer_mt_
- dbo.transfer_mt_
- dbo.transfer_tax_
- dbo.transfer_tax_
- dbo.transfer_tax_
- dbo.transfer_tax_
- dbo.transfer_tax_
- dbo.transfer_tax_
- dbo.transfer_tax_
- dbo.transfer_tax_
- dbo.UDI_owner_t
- dbo.udi_system_s
- dbo.update_agen
- dbo.update_freez
- dbo.update_owne
- dbo.update_owne
- dbo.user_input_q
- dbo.user_input_q
- dbo.user_list_viev
- dbo.user_rights
- dbo.validate_coll_
- dbo.validation_tol
- dbo.value_cert_n
- dbo.value_cert_n
- dbo.value_cert_n
- dbo.visibility_acce
- dbo.vit_sales
- dbo.websessionst
- dbo.workstation_
- dbo.xsp_pacs_co
- dbo.year_to_date
- dbo.year_to_date
- dbo.year_to_date
- dbo.year_to_date
- dbo.year_to_date
- dbo.ZipAssociatio
- Views

pacsserver.pac...SQLQuery1.sql* | Object Explorer Details

```
select * from property_val
```

Results | Messages

	prop_id	prop_val_yr	prop_val	chg...	notice_mail_dt	land_hstd_val	land_non_hstd_val	imprv_hstd_val	imprv_non_hstd_val	appraised_val	assessed_val	market	ag_use_val	ag_ma
1...	12178	1998	NULL	NULL	NULL	0	0	0	0	400	400	13970	400	13970
1...	12179	1998	NULL	NULL	NULL	300	590	22350	128010	155190	155190	232...	3940	80940
1...	12180	1998	NULL	NULL	NULL	590	0	94740	6090	105150	105150	182...	3730	81250
1...	12181	1998	NULL	NULL	NULL	0	0	0	300	1180	1180	15090	880	14790
1...	12182	1998	NULL	NULL	NULL	0	0	0	0	2750	2750	25530	2750	25530
1...	12183	1998	NULL	NULL	NULL	0	0	0	2460	6740	6740	81350	4280	78890
1...	12185	1998	NULL	NULL	NULL	9380	0	2530	0	11910	11910	11910	0	0
1...	12186	1998	NULL	NULL	NULL	3360	0	81520	0	84880	84880	84880	0	0
1...	12187	1998	NULL	NULL	NULL	470	0	62290	10470	77570	77570	133...	4340	59990
1...	12189	1998	NULL	NULL	NULL	14960	0	36820	0	51780	51780	51780	0	0
1...	12190	1998	NULL	NULL	NULL	4830	0	28310	0	33140	33140	33140	0	0
1...	12191	1998	NULL	NULL	NULL	0	4710	0	7950	12660	12660	12660	0	0
1...	12192	1998	NULL	NULL	NULL	0	0	0	3450	15700	15700	102...	12250	98910
1...	12193	1998	NULL	NULL	NULL	0	0	0	0	11150	11150	138...	11150	13800
1...	12194	1998	NULL	NULL	NULL	530	0	7580	0	11380	11380	59650	3270	51540
1...	12195	1998	NULL	NULL	NULL	0	0	0	3370	9020	9020	69430	5650	66060
1...	12196	1998	NULL	NULL	NULL	340	0	6020	0	6360	6360	6360	0	0
1...	12197	1998	NULL	NULL	NULL	340	0	6720	0	7060	7060	7060	0	0
1...	12198	1998	NULL	NULL	NULL	2040	0	75920	0	77960	77960	77960	0	0
1...	12199	1998	NULL	NULL	NULL	0	4970	0	0	4970	4970	4970	0	0
1...	12200	1998	NULL	NULL	NULL	1260	0	31480	300	33590	33590	44380	550	11340
1...	12201	1998	NULL	NULL	NULL	4830	0	23750	0	28580	28580	28580	0	0
1...	12202	1998	NULL	NULL	NULL	0	29490	0	0	30950	30950	57550	1460	28060
1...	12204	1998	NULL	NULL	NULL	32000	0	27180	0	59180	59180	59180	0	0
1...	12205	1998	NULL	NULL	NULL	740	150	49200	10390	64430	64430	116...	3950	56100
1...	12207	1998	NULL	NULL	NULL	8740	0	116070	0	124810	124810	124...	0	0
1...	12209	1998	NULL	NULL	NULL	2810	0	75410	1000	79800	79800	107...	580	28040
1...	12210	1998	NULL	NULL	NULL	0	0	0	0	12880	12880	75620	12880	75620
1...	12211	1998	NULL	NULL	NULL	3150	0	23890	90	31610	31610	80740	4480	53610
1...	12212	1998	NULL	NULL	NULL	0	0	0	0	160	160	2310	160	2310
1...	12213	1998	NULL	NULL	NULL	0	1490	0	0	1490	1490	1490	0	0
1...	12214	1998	NULL	NULL	NULL	6470	0	8530	0	15000	15000	15000	0	0
1...	12215	1998	NULL	NULL	NULL	720	0	58100	0	58820	58820	58820	0	0

Executing query... | pacsserver (9.0 SP4) | LEE\tasupport (63) | pacs_oltp | 00:00:19 | 0 rows

Ready | Ln 1 | Col 9 | Ch 9 | INS

Sorting Retrieved DATA

- Sorting Data
- Sorting by Multiple Columns
- Sorting by Column positions
- Specifying Sort Direction

Sorting Data

- When retrieving data it is displayed in order in which it appears in the underlying tables. This could be in the order in which it is entered into the table. Adding a sort order will allow you do display the results the way that you would like to see them.
- For example:

SELECT **PROP_ID** FROM **PROPERTY_VAL**

ORDER BY PROP_ID

By adding the order by clause, it will sort the data by prop_id in numerical order.

Sorting by Multiple Columns

- Sometimes it is necessary to sort data by more than one column.
- For example you would like to display your properties by market value, then by property id.

```
SELECT PROP_ID,PROP_VAL_YR,MARKET
FROM PROPERTY_VAL
ORDER BY MARKET,PROP_ID
```

- It is important to understand that when you are sorting by multiple columns, the sort sequence is exactly as specified.

Sorting by Column positions

- This sort option works exactly like the previous sort, the difference is that you can type the position number of the your data results instead of typing in the column name.
- For example

```
SELECT PROP_ID,PROP_VAL_YR,MARKET
FROM PROPERTY_VAL
ORDER BY 3,1
```

Specifying Sort Direction

- Data sorting is not limited to ascending sort orders (A to Z). This is defaulted unless otherwise specified. To have your results sort from (Z to A) you will need to add a direction to your order by clause.

```
SELECT PROP_ID,PROP_VAL_YR,MARKET
FROM PROPERTY_VAL
ORDER BY PROP_ID DESC
```

Filtering Data

- WHERE, AND , OR Clauses
- OPERATORS AND WILD CARDS

WHERE, AND , OR Clauses

- **WHERE** - Specifies a search condition to restrict the rows returned.

```
SELECT PROP_ID FROM PROPERTY_VAL
WHERE PROP_VAL_YR = 2011
```

- **AND** - Allows for you to combine Conditions all conditions must be met for the data to be returned

```
SELECT PROP_ID FROM PROPERTY_VAL
WHERE PROP_VAL_YR = 2011
AND  PROP_INACTIVE_DT IS NULL
```

- **OR** - Allows for you to combine conditions where if the date can bet either of the conditions to be returned.

```
SELECT PROP_ID FROM PROPERTY_VAL
WHERE PROP_VAL_YR = 2011
AND  (PROP_INACTIVE_DT IS NULL  OR UDI_PARENT = 'T')
```

OPERATORS AND WILD CARDS

- **EQUALS** - Compares two expressions (a comparison operator).

  Expression = Expression
- **NOT EQUALS** - Tests whether one expression is not equal to another expression (a comparison operator). Functions the same as the Not Equal To (<>) comparison operator.

  Expression <> Expression
- **LESS THAN** Tests whether one expression is less than to another expression (a comparison operator) Functions the same as the Less than (<) comparison operator.

  Expression < Expression
- **LESS THAN OR EQUAL TO** - Tests whether one expression is less than or equal to another expression (a comparison operator). Functions the same as the less than or Equal To (<=) comparison operator.

  Expression <= Expression
- **GREATER THAN** - Tests whether one expression is greater than another expression (a comparison operator). Functions the same as the greater than (>) comparison operator.

  Expression > Expression
- **GREATER THAN OR EQUAL TO** - Tests whether one expression is greater than or equal to another expression (a comparison operator). Functions the same as the greater than or equal (>=) comparison operator.

  Expression >= Expression

OPERATORS AND WILD CARDS

- **LIKE** - Determines whether or not a given character string matches a specified pattern. A pattern can include regular characters and wildcard characters. During pattern matching, regular characters must exactly match the characters specified in the character string. Wildcard characters, however, can be matched with arbitrary fragments of the character string. Using wildcard characters makes the LIKE operator more flexible than using the = and != string comparison operators.

  match_expression **[NOT] LIKE** ***pattern*** **[ESCAPE** ***escape_character*** **]**

- **BETWEEN** - Specifies a range to test.

  test_expression **BETWEEN** ***begin_expression*** **AND** ***end_expression***

- **IN** - Determines if a given value matches any value in a subquery or a list.

  test_expression **IN**
  (
  subquery
  | ***expression*** **[,...*n*]**
)

- **NOT IN** - Determines if a given value matches any value in a subquery or a list.

  test_expression **[NOT] IN**
  (
  subquery
  | ***expression*** **[,...*n*]**
)

HARRIS GOVERN

OPERATORS AND WILD CARDS

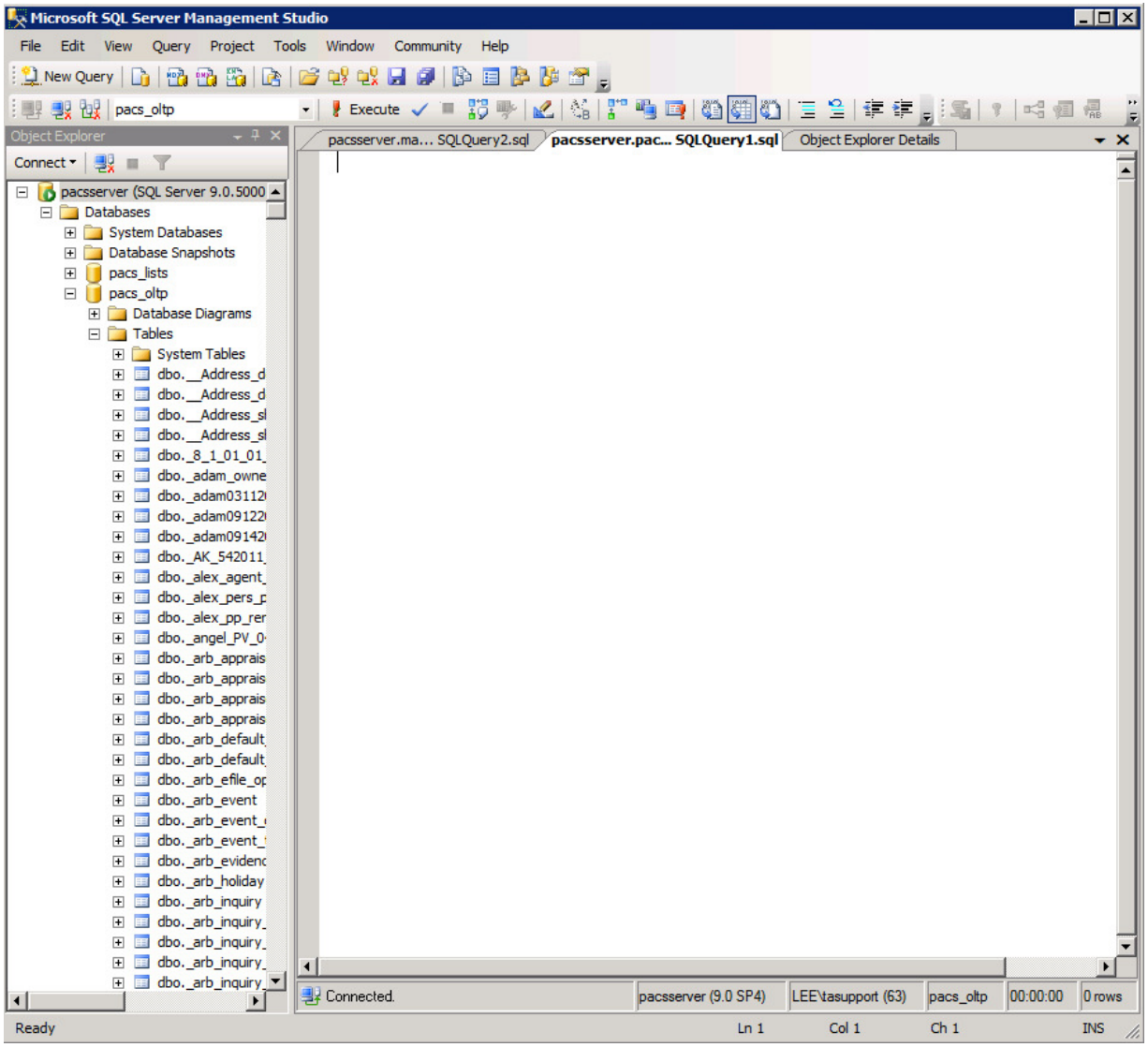
- Wild Cards
  - % is the most frequently used wild card.
  - Can only be used with text fields.
  - Will return based on where the wildcard is placed in the search clause
    - Examples;

WHERE file_as_name LIKE ‘FRANK%’

WHERE file_as_name LIKE ‘%JONES’

WHERE file_as_name LIKE ‘%FRANK%’

SQL Query Window



HARRIS GOVERN